



Transformers documentation

Descripción general de los tokenizadores ▾



Join the Hugging Face community

and get access to the augmented documentation experience

[Sign Up](#)

to get started



Descripción general de los tokenizadores

[Open in Colab](#)[Open](#)[Studio Lab](#)[Copy page](#)

En esta página, veremos más de cerca la tokenización.



Descripción general de los tokenizadores

Hugging Face



0:17 / 0:55



Más videos



Como vimos en [el tutorial de preprocesamiento](#), tokenizar un texto es dividirlo en palabras o subpalabras, que luego se convierten en índices o ids a través de una tabla de búsqueda. Convertir palabras o subpalabras en ids es sencillo, así que en es

[Ask HuggingChat](#)

dividir un texto en palabras o subpalabras (es decir, tokenizar un texto). Más específicamente, examinaremos los tres principales tipos de tokenizadores utilizados en 🧡 Transformers: Byte-Pair Encoding (BPE), WordPiece y SentencePiece, y mostraremos ejemplos de qué tipo de tokenizador se utiliza en cada modelo.

Ten en cuenta que en las páginas de los modelos, puedes ver la documentación del tokenizador asociado para saber qué tipo de tokenizador se utilizó en el modelo preentrenado. Por ejemplo, si miramos BertTokenizer, podemos ver que dicho modelo utiliza WordPiece.

Introducción

Dividir un texto en trozos más pequeños es más difícil de lo que parece, y hay múltiples formas de hacerlo. Por ejemplo, veamos la oración "Don't you love 🧡 Transformers? We sure do."



Word-based tokenizers

Hugging Face



Mirar en

Una forma sencilla de tokenizar este texto es dividirlo por espacios, lo que daría:

```
["Don't", "you", "love", "🧡", "Transformers?", "We", "sure", "do."]
```

Este es un primer paso sensato, pero si miramos los tokens "Transformers?" y "do.". notamos que las puntuaciones están unidas a las palabras "Tra



Deberíamos tener en cuenta la puntuación para que un modelo no tenga que aprender una representación diferente de una palabra y cada posible símbolo de puntuación que podría seguirle, lo que explotaría el número de representaciones que el modelo tiene que aprender. Teniendo en cuenta la puntuación, tokenizar nuestro texto daría:

```
["Don", "'", "t", "you", "love", "😊", "Transformers", "?", "We", "sure", "do", "."]
```

Mejor. Sin embargo, es desventajoso cómo la tokenización trata la palabra "Don't". "Don't" significa "do not", así que sería mejor tokenizada como ["Do", "n't"]. Aquí es donde las cosas comienzan a complicarse, y es la razón por la que cada modelo tiene su propio tipo de tokenizador. Dependiendo de las reglas que apliquemos para tokenizar un texto, se genera una salida tokenizada diferente para el mismo texto. Un modelo preentrenado solo se desempeña correctamente si se le proporciona una entrada que fue tokenizada con las mismas reglas que se utilizaron para tokenizar sus datos de entrenamiento.

spaCy y Moses son dos tokenizadores basados en reglas populares. Al aplicarlos en nuestro ejemplo, *spaCy* y *Moses* generarían algo como:

```
["Do", "n't", "you", "love", "😊", "Transformers", "?", "We", "sure", "do", "."]
```

Como se puede ver, aquí se utiliza tokenización de espacio y puntuación, así como tokenización basada en reglas. La tokenización de espacio y puntuación y la tokenización basada en reglas son ambos ejemplos de tokenización de palabras, que se define de manera simple como dividir oraciones en palabras. Aunque es la forma más intuitiva de dividir textos en trozos más pequeños, este método de tokenización puede generar problemas para corpus de texto masivos. En este caso, la tokenización de espacio y puntuación suele generar un vocabulario muy grande (el conjunto de todas las palabras y tokens únicos utilizados). Ej., Transformer XL utiliza tokenización de espacio y puntuación, lo que resulta en un tamaño de vocabulario de 267,735.

Un tamaño de vocabulario tan grande fuerza al modelo a tener una matriz de embeddings enormemente grande como capa de entrada y salida, lo que causa un aumento tanto en la complejidad de memoria como en la complejidad de tiempo. En general, los modelos de



transformadores rara vez tienen un tamaño de vocabulario mayor que 50,000, especialmente si están preentrenados solo en un idioma.

Entonces, si la simple tokenización de espacios y puntuación es insatisfactoria, ¿por qué no tokenizar simplemente en caracteres?



Character-based tokenizers

Hugging Face



Mirar en

Aunque la tokenización de caracteres es muy simple y reduciría significativamente la complejidad de memoria y tiempo, hace que sea mucho más difícil para el modelo aprender representaciones de entrada significativas. *Ej.* aprender una representación independiente del contexto para la letra "t" es mucho más difícil que aprender una representación independiente del contexto para la palabra "today". Por lo tanto, la tokenización de caracteres suele acompañarse de una pérdida de rendimiento. Así que para obtener lo mejor de ambos mundos, los modelos de transformadores utilizan un híbrido entre la tokenización de nivel de palabra y de nivel de carácter llamada **tokenización de subpalabras**.

Tokenización de subpalabras





Subword-based tokenizers

Hugging Face



Mirar en

Los algoritmos de tokenización de subpalabras se basan en el principio de que las palabras frecuentemente utilizadas no deberían dividirse en subpalabras más pequeñas, pero las palabras raras deberían descomponerse en subpalabras significativas. Por ejemplo, "annoyingly" podría considerarse una palabra rara y descomponerse en "annoying" y "ly". Ambas "annoying" y "ly" como subpalabras independientes aparecerían con más frecuencia al mismo tiempo que se mantiene el significado de "annoyingly" por el significado compuesto de "annoying" y "ly". Esto es especialmente útil en lenguas aglutinantes como el turco, donde puedes formar palabras complejas (casi) arbitrariamente largas concatenando subpalabras.

La tokenización de subpalabras permite al modelo tener un tamaño de vocabulario razonable mientras puede aprender representaciones contextuales independientes significativas. Además, la tokenización de subpalabras permite al modelo procesar palabras que nunca ha visto antes, descomponiéndolas en subpalabras conocidas. Por ejemplo, el tokenizador [BertTokenizer](#) tokeniza "I have a new GPU!" de la siguiente manera:

```
>>> from transformers import BertTokenizer

>>> tokenizer = BertTokenizer.from_pretrained("google-bert/bert-base-uncased")
>>> tokenizer.tokenize("I have a new GPU!")
["i", "have", "a", "new", "gp", "t#fu", "!"]
```



Debido a que estamos considerando el modelo sin mayúsculas, la oración se convirtió a minúsculas primero. Podemos ver que las palabras ["i", "have", "a", "new"] están presentes en el vocabulario del tokenizador, pero la palabra "gpu" no. En consecuencia, el tokenizador divide "gpu" en subpalabras conocidas: ["gp" y "##u"]. "##" significa que el resto del token debería adjuntarse al anterior, sin espacio (para decodificar o revertir la tokenización).

Como otro ejemplo, el tokenizador [XLNetTokenizer](#) tokeniza nuestro texto de ejemplo anterior de la siguiente manera:

```
>>> from transformers import XLNetTokenizer

>>> tokenizer = XLNetTokenizer.from_pretrained("xlnet/xlnet-base-cased")
>>> tokenizer.tokenize("Don't you love 😊 Transformers? We sure do.")
["_Don", "'", "t", "_you", "_love", "_", "😊", "_", "Transform", "ers", "?", "_We", "_su"]
```

Hablaremos del significado de esos "_" cuando veamos [SentencePiece](#). Como se puede ver, la palabra rara "Transformers" se ha dividido en las subpalabras más frecuentes "Transform" y "ers".

Ahora, veamos cómo funcionan los diferentes algoritmos de tokenización de subpalabras. Ten en cuenta que todos esos algoritmos de tokenización se basan en alguna forma de entrenamiento que usualmente se realiza en el corpus en el que se entrenará el modelo correspondiente.

Byte-Pair Encoding (BPE)

La Codificación por Pares de Bytes (BPE por sus siglas en inglés) fue introducida en [Neural Machine Translation of Rare Words with Subword Units \(Sennrich et al., 2015\)](#). BPE se basa en un pre-tokenizador que divide los datos de entrenamiento en palabras. La pre-tokenización puede ser tan simple como la tokenización por espacio, por ejemplo, [GPT-2](#), [RoBERTa](#). La pre-tokenización más avanzada incluye la tokenización basada en reglas, por ejemplo, [XLM](#), [FlauBERT](#) que utiliza Moses para la mayoría de los idiomas, o [GPT](#) que utiliza spaCy y ftfy, para contar la frecuencia de cada palabra en el corpus de entrenamiento.



Después de la pre-tokenización, se ha creado un conjunto de palabras únicas y ha determinado la frecuencia con la que cada palabra apareció en los datos de entrenamiento. A continuación, BPE crea un vocabulario base que consiste en todos los símbolos que aparecen en el conjunto de palabras únicas y aprende reglas de fusión para formar un nuevo símbolo a partir de dos símbolos del vocabulario base. Lo hace hasta que el vocabulario ha alcanzado el tamaño de vocabulario deseado. Tenga en cuenta que el tamaño de vocabulario deseado es un hiperparámetro que se debe definir antes de entrenar el tokenizador.

Por ejemplo, supongamos que después de la pre-tokenización, se ha determinado el siguiente conjunto de palabras, incluyendo su frecuencia:

```
("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
```

En consecuencia, el vocabulario base es ["b", "g", "h", "n", "p", "s", "u"]. Dividiendo todas las palabras en símbolos del vocabulario base, obtenemos:

```
("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)
```

Luego, BPE cuenta la frecuencia de cada par de símbolos posible y selecciona el par de símbolos que ocurre con más frecuencia. En el ejemplo anterior, "h" seguido de "u" está presente $10 + 5 = 15$ veces (10 veces en las 10 ocurrencias de "hug", 5 veces en las 5 ocurrencias de "hugs"). Sin embargo, el par de símbolos más frecuente es "u" seguido de "g", que ocurre $10 + 5 + 5 = 20$ veces en total. Por lo tanto, la primera regla de fusión que aprende el tokenizador es agrupar todos los símbolos "u" seguidos de un símbolo "g" juntos. A continuación, "ug" se agrega al vocabulario. El conjunto de palabras entonces se convierte en

```
("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)
```

Seguidamente, BPE identifica el próximo par de símbolos más común. Es "u" seguido de "n", que ocurre 16 veces. "u", "n" se fusionan en "un" y se agregan al vocabulario. El próximo par de símbolos más frecuente es "h" seguido de "ug", que ocurre 15 veces. De nuevo, el par se fusiona y "hug" se puede agregar al vocabulario.



En este momento, el vocabulario es ["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"] y nuestro conjunto de palabras únicas se representa como:

```
("hug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("hug" "s", 5)
```

Suponiendo que el entrenamiento por Byte-Pair Encoding se detuviera en este punto, las reglas de combinación aprendidas se aplicarían entonces a nuevas palabras (siempre que esas nuevas palabras no incluyan símbolos que no estuvieran en el vocabulario base). Por ejemplo, la palabra "bug" se tokenizaría como ["b", "ug"], pero "mug" se tokenizaría como ["<unk>", "ug"] ya que el símbolo "m" no está en el vocabulario base. En general, las letras individuales como "m" no se reemplazan por el símbolo "<unk>" porque los datos de entrenamiento usualmente incluyen al menos una ocurrencia de cada letra, pero es probable que suceda para caracteres especiales como los emojis.

Como se mencionó anteriormente, el tamaño del vocabulario, es decir, el tamaño del vocabulario base + el número de combinaciones, es un hiperparámetro que se debe elegir. Por ejemplo, GPT tiene un tamaño de vocabulario de 40,478 ya que tienen 478 caracteres base y eligieron detener el entrenamiento después de 40,000 combinaciones.

Byte-level BPE

Un vocabulario base que incluya todos los caracteres base posibles puede ser bastante extenso si, por ejemplo, se consideran todos los caracteres unicode como caracteres base. Para tener un vocabulario base mejor, GPT-2 utiliza bytes como vocabulario base, lo que es un truco astuto para forzar el vocabulario base a ser de tamaño 256 mientras se asegura de que cada carácter base esté incluido en el vocabulario. Con algunas reglas adicionales para tratar con la puntuación, el tokenizador de GPT2 puede tokenizar cualquier texto sin la necesidad del símbolo <unk>. GPT-2 tiene un tamaño de vocabulario de 50,257, lo que corresponde a los 256 tokens base de bytes, un token especial de fin de texto y los símbolos aprendidos con 50,000 combinaciones.

WordPiece

WordPiece es el algoritmo de tokenización de subpalabras utilizado por BERT, DistilBERT, FLAN, etc. El algoritmo fue descrito en Japanese and Korean Voice

similar a BPE. WordPiece inicializa el vocabulario para incluir cada carácter presente en los datos de entrenamiento y aprende progresivamente un número determinado de reglas de fusión. A diferencia de BPE, WordPiece no elige el par de símbolos más frecuente, sino el que maximiza la probabilidad de los datos de entrenamiento una vez agregado al vocabulario.

¿Qué significa esto exactamente? Refiriéndonos al ejemplo anterior, maximizar la probabilidad de los datos de entrenamiento es equivalente a encontrar el par de símbolos cuya probabilidad dividida entre las probabilidades de su primer símbolo seguido de su segundo símbolo es la mayor entre todos los pares de símbolos. Ej. "u" seguido de "g" solo habría sido combinado si la probabilidad de "ug" dividida entre "u" y "g" habría sido mayor que para cualquier otro par de símbolos. Intuitivamente, WordPiece es ligeramente diferente a BPE en que evalúa lo que *pierde* al fusionar dos símbolos para asegurarse de que *valga la pena*.

Unigram

Unigram es un algoritmo de tokenización de subpalabras introducido en [Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates \(Kudo, 2018\)](#). A diferencia de BPE o WordPiece, Unigram inicializa su vocabulario base con un gran número de símbolos y progresivamente recorta cada símbolo para obtener un vocabulario más pequeño. El vocabulario base podría corresponder, por ejemplo, a todas las palabras pre-tokenizadas y las subcadenas más comunes. Unigram no se utiliza directamente para ninguno de los modelos transformers, pero se utiliza en conjunto con [SentencePiece](#).

En cada paso de entrenamiento, el algoritmo Unigram define una pérdida (a menudo definida como la probabilidad logarítmica) sobre los datos de entrenamiento dados el vocabulario actual y un modelo de lenguaje unigram. Luego, para cada símbolo en el vocabulario, el algoritmo calcula cuánto aumentaría la pérdida general si el símbolo se eliminara del vocabulario. Luego, Unigram elimina un porcentaje p de los símbolos cuyo aumento de pérdida es el más bajo (siendo p generalmente 10% o 20%), es decir, aquellos símbolos que menos afectan la pérdida general sobre los datos de entrenamiento. Este proceso se repite hasta que el vocabulario haya alcanzado el tamaño deseado. El algoritmo Unigram siempre mantiene los caracteres base para que cualquier palabra pueda ser tokenizada.



Debido a que Unigram no se basa en reglas de combinación (en contraste con BPE y WordPiece), el algoritmo tiene varias formas de tokenizar nuevo texto después del entrenamiento. Por ejemplo, si un tokenizador Unigram entrenado exhibe el vocabulario:

```
["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"],
```

"hugs" podría ser tokenizado tanto como ["hug", "s"], ["h", "ug", "s"] o ["h", "u", "g", "s"] . ¿Cuál elegir? Unigram guarda la probabilidad de cada token en el corpus de entrenamiento junto con el vocabulario, para que la probabilidad de que cada posible tokenización pueda ser computada después del entrenamiento. El algoritmo simplemente elige la tokenización más probable en la práctica, pero también ofrece la posibilidad de muestrear una posible tokenización según sus probabilidades.

Esas probabilidades están definidas por la pérdida en la que se entrena el tokenizador. Suponiendo que los datos de entrenamiento constan de las palabras $x_1, \dots, x_N$ y que el conjunto de todas las posibles tokenizaciones para una palabra x_i se define como $S(x_i)$, entonces la pérdida general se define como:

$$\mathcal{L} = - \sum_{i=1}^N \log \left(\sum_{x \in S(x_i)} p(x) \right)$$

SentencePiece

Todos los algoritmos de tokenización descritos hasta ahora tienen el mismo problema: se asume que el texto de entrada utiliza espacios para separar palabras. Sin embargo, no todos los idiomas utilizan espacios para separar palabras. Una posible solución es utilizar pre-tokenizadores específicos del idioma, *ej.* XLM utiliza un pre-tokenizador específico para chino, japonés y tailandés. Para resolver este problema de manera más general, SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing (Kudo et al., 2018) trata el texto de entrada como una corriente de entrada bruta, por lo que incluye el espacio en el conjunto de caracteres para utilizar. Luego utiliza el algoritmo BPE o unigram para construir el vocabulario apropiado.



Por ejemplo, [XLNetTokenizer](#) utiliza SentencePiece, razón por la cual en el ejemplo anterior se incluyó el carácter "_" en el vocabulario. Decodificar con SentencePiece es muy fácil, ya que todos los tokens pueden simplemente concatenarse y "_" se reemplaza por un espacio.

Todos los modelos transformers de nuestra biblioteca que utilizan SentencePiece lo utilizan en combinación con Unigram. Ejemplos de los modelos que utilizan SentencePiece son [ALBERT](#), [XLNet](#), [Marian](#) y [T5](#).

[Update on GitHub](#)

← Como los 🧠 Transformers resuelven tareas

Mecanismos de atención →

